

hyper2kvm

Enterprise-Grade VM Migration Toolkit

Any Hypervisor → KVM with Zero-Downtime & Automated Fixes

Susant Sahani

Python 3.10+ | LGPL-3.0 | PyPI: [hyper2kvm](#)

The Problem

Migrating VMs between hypervisors is **painful and error-prone**:

- **Disk format incompatibility** — VMDK, VHDX, VHD, OVA all need conversion
- **Boot failures** — wrong drivers, broken GRUB, stale fstab entries
- **Manual intervention** — trial-and-error fixing after each attempt
- **No Windows support** — VirtIO drivers must be injected manually
- **No automation** — each VM is a one-off effort

Traditional approach:

Convert → Boot →
Hope

✗ ~40% first-boot failure rate

The Solution: hyper2kvm

Deterministic offline fixes ensure **first-boot success** — every time.

Traditional Tools	hyper2kvm
✗ Convert disk format only	✓ Convert + deep inspection + repair
✗ "Boot and hope"	✓ Deterministic first-boot success
✗ Manual fixes required	✓ Automated bootloader, fstab, driver fixes
✗ Trial and error	✓ Validated pipeline with smoke tests
✗ Linux only	✓ Linux + Windows (35+ OS versions)
✗ Single VM at a time	✓ Batch processing + K8s deployment

At a Glance

480+

VMCraft APIs

8

Input Formats

35+

OS Versions

380+

Test Suite

5-7x

Faster Performance

0.71s

LVM Activation

100%

Host Protection

0

C Dependencies

VMDK

OVA

OVF

VHD

VHDX

AMI

Azure VHD

Raw

→

KVM

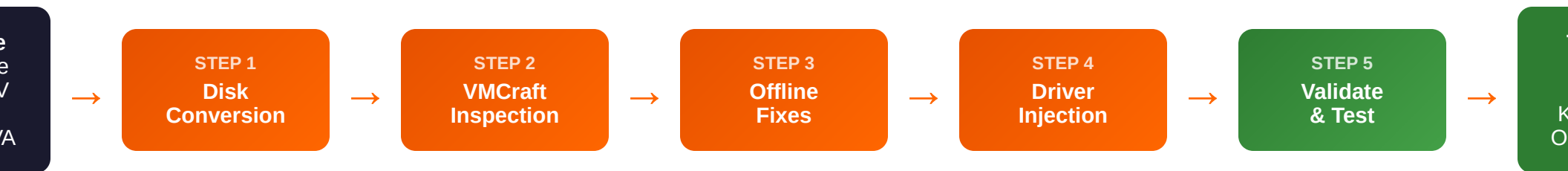
libvirt

KubeVirt

OpenShift

k3s

Architecture & Pipeline



What Happens in Offline Fixes

Bootloader

GRUB/GRUB2 repair for VirtIO disk controllers. Ensures kernel boots with correct root device.

Filesystem

fstab stabilization (UUID/PARTUUID), initramfs regeneration with VirtIO drivers.

Networking

Interface config adaptation, udev rules cleanup, NetworkManager/systemd-networkd setup.

VMCraft Engine

Pure Python VM manipulation engine — 480+ API methods, zero C dependencies.

Performance vs libguestfs

Launch Time

libguestfs: 9.7s

**VMCraft:
1.9s**

Memory Usage

libguestfs: 280 MB

VMCraft: 95 MB

System Calls

libguestfs: 14,200

VMCraft: 8,500

Code Example

```
from hyper2kvm.core.vmcraft import VMCraft

with VMCraft() as g:
    g.add_disk("/vms/server.qcow2")
    g.launch() # ~1.9 seconds

    # Inspect OS
    os_info = g.inspect_os()
    print(f"OS: {os_info.product_name}")

    # Read/write files
    g.write("/etc/motd",
           "Migrated to KVM!\n")

    # Partition & LVM management
    parts = g.list_partitions()
    vgs = g.vglist()
```

Key Features

Automated Offline Fixes

- **GRUB/GRUB2** bootloader repair
- **fstab** UUID stabilization
- **initramfs** regeneration with VirtIO
- **Network** interface adaptation
- **SELinux** relabeling
- **LVM** sandboxed activation

Live Fix (Zero Downtime)

- SSH-based remote fixes
- No shutdown required
- All fix types supported

Windows Migration

- **VirtIO injection** — storage, network, balloon, serial
- **Registry** — boot-critical driver registration
- **License** — OEM, Retail, MAK, KMS
- **Active Directory** — domain rejoin
- **App compat** — SQL Server, hardware-dependent apps

Win 7-11

Server 2008 R2-2025

x64

x86

Validation & Testing

- **QEMU smoke test** — automated boot verification
- **libvirt test** — full VM lifecycle test
- **Health checks** — post-migration validation
- **Checksum** — SHA256 integrity

Batch Processing

- Parallel multi-VM migration
- Manifest-based orchestration
- Progress tracking & reporting
- Rollback support

Supported Platforms

Linux Distributions

Family	Versions
RHEL / CentOS	7, 8, 9
Fedora	Latest releases
Ubuntu	18.04 – 24.04
Debian	10, 11, 12
SUSE / openSUSE	15.x, Tumbleweed
Rocky / Alma Linux	8, 9
Oracle Linux	7, 8, 9
Amazon Linux	2, 2023
Photon OS	3, 4, 5
Flatcar / CoreOS	Latest

Windows Versions

Edition	Versions
Windows Desktop	7, 8, 8.1, 10, 11
Windows Server	2008 R2 – 2025
Architecture	x64 and x86

Deployment Targets

libvirt / KVM

KubeVirt

Kubernetes

OpenShift

k3s

Migration Workflow

Step 1: Configure (YAML)

```
cmd: local
vmdk: /path/to/server.vmdk
output_dir: /output
to_output: server.qcow2
out_format: qcow2
fstab_mode: stabilize-all
regen_initramfs: true
no_grub: false
libvirt_test: true
```

Step 2: Run

```
# One command migration
sudo h2kvmctl \
  --config migration.yaml
```

Step 3: Deploy

```
# Deploy to libvirt
sudo virsh define server.xml
sudo virsh start server

# Verify
sudo virsh domifaddr server
```

Or: Direct to K8s

```
# One-command K8s deploy
sudo h2kvmctl \
  --config migration.yaml \
  --deploy-k8s \
  --k8s-namespace prod \
  --k8s-vm-name web-01 \
  --k8s-auto-start
```

Automated:

1. Convert + fix
2. Upload to PVC
3. Create KubeVirt VM
4. Start & verify

Enterprise Features

Security

- **LUKS / TPM2** encryption support
- **Host VG protection** — 100% sandboxed LVM
- **Secure boot** compatibility
- **SELinux** relabeling on migration

LVM Safety

Problem: LVM activation can corrupt host VGs
Solution: Sandboxed activation with auto host-exclusion
Result: 0.71s activation, 7x faster, 100% safe

Daemon Mode

- systemd service integration
- REST API for automation
- Metrics and monitoring

Kubernetes / OpenShift

- One-command K8s deployment
- PVC creation & upload
- KubeVirt VM provisioning
- OperatorHub integration
- SecurityContextConstraints
- Routes & OAuth
- OLM bundle packaging

Observability

- **TUI dashboard** — Textual / Curses / CLI
- **Progress bars** with ETA tracking
- **Debug bundles** for troubleshooting
- **Analytics** & reporting

Systemd Integration

- 20 integrated systemd tools
- Platform detection, disk inspection
- Resource control, multi-stage testing

Getting Started

Installation

```
# Install with all features
pip install "hyper2kvm[full]"

# Or minimal install
pip install hyper2kvm
```

System Dependencies (Optional)

```
# Fedora / RHEL
sudo dnf install -y \
    qemu-img qemu-system-x86

# Ubuntu / Debian
sudo apt-get install -y \
    qemu-utils qemu-system-x86

# Windows migration support
sudo dnf install -y \
    ntfs-3g libhivex-bin
```

CLI Commands

```
# Interactive CLI (recommended)
h2kvmctl --version

# Daemon / systemd mode
hyper2kvm --version

# Both are identical
```

Project Structure

```
hyper2kvm/
cli/          # Click-based CLI
core/         # VMCraft engine
converters/   # qemu-img wrappers
fixers/       # Offline fix modules
  bootloader/ # GRUB repair
  windows/    # VirtIO, registry
  live/       # SSH-based fixes
providers/    # VMware, Azure
orchestration/ # Batch, manifest
infrastructure/ # K8s deployers
systemd/      # systemd tools
luks/         # Encryption
```

hyper2kvm

"Migrate Once, Boot Right."

Deterministic	No more "boot and hope"
Comprehensive	35+ OS versions, 8 input formats
Fast	5-7x faster than traditional tools
Safe	100% host protection, sandboxed ops
Automated	One command: source → running VM
Cloud-native	Kubernetes, OpenShift, KubeVirt
Pure Python	Zero C dependencies, runs anywhere

PyPI: pypi.org/project/hyper2kvm | **GitHub:** github.com/ssahani/hyper2kvm | **License:** LGPL-3.0